# Module 36

## Algorithms and Data Structures/1: Algorithms and Complexity Analysis

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
ppd@cse.iitkgp.ac.in

March 17, 2026

# Week Recap

- Had a glimpse of Application Programs across various sectors
- Understood the architectures for an application and their classification and evolution
- Glimpsed at architecture for a few sample applications
- Familiarized with the Fundamentals notions and technologies of Web
- Learnt about Scripting and the notions of Servlets
- Learnt to use SQL from a programming language
- Learnt to build Python Web Applications with PostgreSQL using psycopg2 and Flask
- Understood the steps in the Rapid Application Development Process
- Exposed to the issues in Application Performance and Security
- Learnt the distinctive features of Mobile Apps

# Module Objectives

- Define Algorithms and its difference with Programs
- Analyze algorithms for performance of time, space, power, etc.
- Introduce Asymptotic notation for representation of complexity
- Consider complexity of common algorithms

# Module Outline

- Algorithms and Programs
- Analysis of Algorithms
- Complexity Chart

# Algorithms and Programs

# Algorithms and Programs

## Algorithm

- An algorithm is a finite sequence of well-defined , computer-implementable (optional) instructions, typically to solve a class of specific problems or to perform a computation.
- Algorithms are always unambiguous and are used as specifications for performing calculations, data processing, automated reasoning, and other tasks.
- An algorithm must terminate

## Program

- A computer program is a collection of instructions that can be executed by a computer to perform a specific task
- A computer program is usually written by a computer programmer in a programming language.
- A programs implements an algorithm
- A program may or may not terminate. For example, an OS

# Analysis of Algorithms

# Analysis of Algorithms

- **Why?**
  - Set the motivation for algorithm analysis: Why analyze?
- **What?**
  - Identify what all need to be analyzed: What to analyze?
- **How?**
  - Learn the techniques for analysis: How to analyze?
- **Where?**
  - Understand the scenarios for application: Where to analyze?
- **When?**
  - Realize your position for seeking the analysis: When to analyze?

# Why analyze?

## Practical reasons:

- Resources are scarce
- Greed to do more with less
- Avoid performance bugs

## Core Issues:

- Predict performance
  - How much time does binary search take?
- Compare algorithms
  - How quick is Quicksort?
- Provide guarantees
  - Size notwithstanding, Red-Black tree inserts in $\mathcal{O}(\log n)$
- Understand theoretical basis
  - Sorting by comparison cannot do better than $\Omega(n \log n)$

# What to analyze?

**Core Issue: Cannot control what we cannot measure**

- **Time**
  - Story starts here with Analytical Engine
  - Most common analysis factor
  - Representative of various related analysis factors like Power, Bandwidth, Processors
  - Supported by Complexity Classes
- **Space**
  - Widely explored
  - Important for hand-held devices
  - Supported by Complexity Classes

# What to analyze? (2)

## Sum of Natural Numbers

```
int sum(int n) {
    int s = 0;
    for(; n > 0; --n)
        s = s + n;
    return s;
}
```

## Analysis

- Time $T(n) = n$ (additions)
- Space $S(n) = 2$ $(n, s)$

# What to analyze? (3)

## Find a character in a string

```
int find(char *str, char c) {
    for(int i = 0; i < strlen(str); ++i)
        if (str[i] == c)
            return i;
    return 0;
}
```

$n = \text{strlen}(str)$

## Analysis

- Time $T(n) = n\ (\text{compare}) + n * T(\text{strlen}(str)) \approx n + n^2 \approx n^2$
- Space $S(n) = 3\ (str, c, i)$

# What to analyze? (4)

## Minimum of a Sequence of Numbers

```
int min(int a[], int n) {
    for(int i = 0; i < n; ++i) cin >> a[i];
    int t = a[--n];
    for(; n > 0; --n)
        if (t < a[--n])
            t = a[n];
    return t;
}
```

## Analysis

- Time $T(n) = n - 1$ (comparison of value)
- Space $S(n) = n + 3$ ($a[]$'s, $n, i, t$)

# What to analyze? (5)

## Minimum of a Sequence of Numbers 2

```
int min(int n) {
    int x;
    cin >> x;
    int t = x;
    for(; n > 1; --n) {
        cin >> x;
        if (t < x)
            t = x;
    }
    return t;
}
```

## Analysis

- Time $T(n) = n - 1$ (comparison of value)

# How to analyze?

- Counting Models
- Asymptotic Analysis
- Generating Functions
- Master Theorem

# How to analyze?: Counting Models

## Counting Models

- **Core Idea:** Total running time = Sum of cost $\times$ frequency for all operations
- Need to analyze program to determine set of operations
- Cost depends on machine, compiler
- Frequency depends on algorithm, input data
- Machine Model: Random Access Machine (RAM) Computing Model
- Input data & size
- Operations
- Intermediate Stages
- Output data & size

# How to analyze?: Counting Models (2)

## Factorial (Recursive)

```
int fact(int n) {
    if (0 != n) return n*fact(n-1);
    return 1;
}
```

- Time $T(n) = n - 1$ (multiplication)
- Space $S(n) = n + 1$ ($n$'s in recursive calls)

## Factorial (Iterative)

```
int fact(int n) {
    int t = 1;
    for(; n > 0; --n)
        t = t * n;
    return t;
```

# How to analyze?: Asymptotic Analysis

## Asymptotic Analysis

- **Core Idea:** Cannot compare actual times; hence compare Growth or how time increases with input size
- Function Approximation (tilde ($\sim$) notation)
- Common Growth Functions
- Big-Oh ($\mathcal{O}(.)$), Big-Omega ($\Omega(.)$), and Big-Theta ($\Theta(.)$) Notations
- Solve recurrence with Growth Functions

# How to analyze?: Asymptotic Analysis (2)

```
int count = 0;
for (int i = 0; i < N; i++)
    for (int j = i+1; j < N; j++)
        if (a[i] + a[j] == 0)
            count++;
```

## Function Approximation (tilde ($\sim$) notation)

| Operation | Frequency | Approximation |
|---|---|---|
| variable declaration | $N + 2$ | $\sim N$ |
| assignment statement | $N + 2$ | $\sim N$ |
| less than compare | $\frac{1}{2}(N+1)(N+2)$ | $\sim \frac{1}{2}N^2$ |
| equal to compare | $\frac{1}{2}N(N-1)$ | $\sim \frac{1}{2}N^2$ |
| array access | $N(N-1)$ | $\sim N^2$ |

# How to analyze?: Asymptotic Analysis (3)

## Common order-of-growth classifications

| order of growth | name | typical code framework | description |
|---|---|---|---|
| $1$ | constant | `a = b + c;` | statement |
| $\log N$ | logarithmic | `while (N > 1) N = N / 2;` | divide in half |
| $N$ | linear | `for (int i=0; i<N; i++)` | loop |
| $N \log N$ | linearithmic | mergesort | divide and conquer |
| $N^2$ | quadratic | `for (i=0; i<N; i++) for (j=0; j<N; j++)` | double loop |
| $N^3$ | cubic | `for (i=0; i<N; i++) for (j=0; j<N; j++) for (k=0; k<N; k++)` | triple loop |
| $2^N$ | exponential | combinatorial search | exhaustive search |

# Asymptotic notation

For a given function $g(n)$, we denote by $\mathcal{O}(g(n))$ the set of functions:

$$\mathcal{O}(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n), \text{ for all } n > n_0\}$$

- We use $\mathcal{O}$-notation to give an upper bound on a function, to within a constant factor.
- When we say that the running time of A is $\mathcal{O}(n^2)$, we mean that there is a function $f(n)$ that is $\mathcal{O}(n^2)$ such that for any value of $n$, no matter what particular input of size $n$ is chosen, the running time on that input is bounded from above by the value $f(n)$.
- Equivalently, we mean that the worst-case running time is $\mathcal{O}(n^2)$.

# Where to analyze?

## Algorithmic Situation

**Core Idea:** Identify data configurations or scenarios for analysis

- **Best Case** ▷ Minimum running time on an input
- **Worst Case** ▷ Running time guarantee for any input of size $n$
- **Average Case** ▷ Expected running time for a random input of size $n$
- **Probabilistic Case** ▷ Expected running time of a randomized algorithm
- **Amortized Case** ▷ Worst case running time for any sequence of $n$ operations

# Big-O Algorithm Complexity Cheat Sheet

## Common Data Structure Operations

| Data Structure | Time Complexity | | | | | | | | Space Complexity |
|---|---|---|---|---|---|---|---|---|---|
| | Average | | | | Worst | | | | Worst |
| | Access | Search | Insertion | Deletion | Access | Search | Insertion | Deletion | |
| Array | Θ(1) | Θ(n) | Θ(n) | Θ(n) | O(1) | O(n) | O(n) | O(n) | O(n) |
| Stack | Θ(n) | Θ(n) | Θ(1) | Θ(1) | O(n) | O(n) | O(1) | O(1) | O(n) |
| Queue | Θ(n) | Θ(n) | Θ(1) | Θ(1) | O(n) | O(n) | O(1) | O(1) | O(n) |
| Singly-Linked List | Θ(n) | Θ(n) | Θ(1) | Θ(1) | O(n) | O(n) | O(1) | O(1) | O(n) |
| Doubly-Linked List | Θ(n) | Θ(n) | Θ(1) | Θ(1) | O(n) | O(n) | O(1) | O(1) | O(n) |
| Skip List | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | O(n) | O(n) | O(n) | O(n) | O(n log(n)) |
| Hash Table | N/A | Θ(1) | Θ(1) | Θ(1) | N/A | O(n) | O(n) | O(n) | O(n) |
| Binary Search Tree | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | O(n) | O(n) | O(n) | O(n) | O(n) |
| Cartesian Tree | N/A | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | N/A | O(n) | O(n) | O(n) | O(n) |
| B-Tree | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | O(log(n)) | O(log(n)) | O(log(n)) | O(log(n)) | O(n) |
| Red-Black Tree | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | O(log(n)) | O(log(n)) | O(log(n)) | O(log(n)) | O(n) |
| Splay Tree | N/A | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | N/A | O(log(n)) | O(log(n)) | O(log(n)) | O(n) |

# Big-O Algorithm Complexity Cheat Sheet (2)

## Array Sorting Algorithms

| Algorithm | Time Complexity | | | Space Complexity |
|---|---|---|---|---|
| | Best | Average | Worst | Worst |
| Quicksort | Ω(n log(n)) | Θ(n log(n)) | O(n^2) | O(log(n)) |
| Mergesort | Ω(n log(n)) | Θ(n log(n)) | O(n log(n)) | O(n) |
| Timsort | Ω(n) | Θ(n log(n)) | O(n log(n)) | O(n) |
| Heapsort | Ω(n log(n)) | Θ(n log(n)) | O(n log(n)) | O(1) |
| Bubble Sort | Ω(n) | Θ(n^2) | O(n^2) | O(1) |
| Insertion Sort | Ω(n) | Θ(n^2) | O(n^2) | O(1) |
| Selection Sort | Ω(n^2) | Θ(n^2) | O(n^2) | O(1) |
| Tree Sort | Ω(n log(n)) | Θ(n log(n)) | O(n^2) | O(n) |
| Shell Sort | Ω(n log(n)) | Θ(n(log(n))^2) | O(n(log(n))^2) | O(1) |
| Bucket Sort | Ω(n+k) | Θ(n+k) | O(n^2) | O(n) |
| Radix Sort | Ω(nk) | Θ(nk) | O(nk) | O(n+k) |
| Counting Sort | Ω(n+k) | Θ(n+k) | O(n+k) | O(k) |
| Cubesort | Ω(n) | Θ(n log(n)) | O(n log(n)) | O(n) |

# Module Summary

- Need for analyzing the running-time and space requirements of a program
- Asymptotic growth rate or order of the complexity of different algorithms
- Worst-case, average-case and best-case analysis

*Slides used in this presentation are borrowed from* `http://db-book.com/` *with kind permission of the authors.*
*Edited and new slides are marked with 'PPD'.*